

# CARBONROUTE

A Reproducible Benchmark and Deadline-Risk Calibrator for  
Carbon-Aware Batch Scheduling under Forecast Error

**Software Engineering - UCS503**

Submitted to: Sukhpal Singh

**Team: Yuvika Nagpal, Kumkum Gupta, Aaneya Sabharwal**

**Date: 17 August 2026**

Department of Computer Science & Engineering

## 1. Problem Statement

# Why CarbonRoute? Problem & Motivation

- > Cloud workloads such as ML training, data processing, backups, and rendering can be delayed within a deadline.
- > Carbon-aware scheduling attempts to move these workloads toward periods or regions with lower predicted carbon intensity.
- > However, real-world carbon forecasts are noisy and cloud capacity/spot availability changes dynamically.
- > Blindly scheduling based on point forecasts leads to severe deadline violations when forecasts err.
- > CarbonRoute evaluates how reliable scheduling decisions remain under forecast and capacity uncertainty.

# Core Research Question & 10 Objectives

How reliable is carbon-aware scheduling when carbon forecasts, capacity and prices are uncertain, and can deadline-miss risk be calibrated?

- > 1. Build a simulator representing cloud regions, workloads, carbon intensity, cost and capacity.
- > 2. Implement baseline scheduling policies (Immediate, EDF, Cost-Aware, Deterministic Carbon, Baseline).
- > 3. Model forecast uncertainty and error distributions across multiple look-ahead horizons.
- > 4. Develop uncertainty-aware scheduling optimizing  $P(\text{deadline violation} \mid \text{decision}) \leq \tau$ .
- > 5. Evaluate risk calibration using Brier score and Reliability Diagrams.
- > 6. Perform systematic stress testing under extreme grid volatility and capacity contention.
- > 7. Build a reproducible evaluation benchmark with versioned trace datasets.
- > 8. Expose scheduling algorithms via FastAPI REST endpoints and an interactive dashboard.
- > 9. Connect a containerized workload connector (Kubernetes Jobs) for real execution.
- > 10. Deliver a production-ready, open-source research artifact across the 17-week semester.

### 3. Project Scope

# Software Engineering Project Scope

## INCLUDED IN PROJECT

- [+] Cloud & Region Simulator (discrete-event)
- [+] Versioned Traces & Workload Generator
- [+] 6+ Scheduling Policy Implementations
- [+] Forecast Uncertainty & Error Modeller
- [+] Deadline-Risk Calibrator (Brier / ECE)
- [+] Parametric Stress Testing Engine
- [+] FastAPI REST Backend + React Dashboard
- [+] Reproducible Benchmark Framework
- [+] One Containerized Workload Connector

## NOT INCLUDED INITIALLY

- [-] Physical data-center hardware management
- [-] Real-time hypervisor level VM live-migration
- [-] Full proprietary AWS/GCP cloud replacement
- [-] Opaque LLM-based black-box scheduling
- [-] Direct power-grid substation control

#### 4. System Architecture

## End-to-End Modular Architecture

- > [Workload Generator] -> Defines synthetic & trace-based container batch jobs with arrival & deadline.
- > [Cloud Region Simulator] -> Models multi-region electricity carbon intensity (gCO<sub>2</sub>eq/kWh) and capacity.
- > [Trace Builder] -> Standardizes historical forecasts, realized ground-truth, and regional spot tariffs.
- > [Scheduling Engine] -> Compares Immediate, EDF, Cost-Aware, Carbon-Aware, and Uncertainty-Aware policies.
- > [Risk & Uncertainty Model] -> Computes P(deadline miss) using calibrated probability distributions.
- > [Experiment & Stress-Test Engine] -> Evaluates sensitivity across forecast error variances and load spikes.
- > [API & Dashboard] -> FastAPI backend + Modern React frontend providing visual insights and dispatch triggers.

5. Scheduling Policies

# Comparative Scheduling Policies Matrix

|                                 |                                                                                |
|---------------------------------|--------------------------------------------------------------------------------|
| Immediate                       | Executes jobs immediately upon arrival with zero intentional delay.            |
| Earliest Deadline First (EDF)   | Prioritizes jobs with the closest deadline to minimize overdue probability.    |
| Cost-Aware                      | Schedules jobs during lowest cloud electricity/spot price intervals.           |
| Deterministic Carbon-Aware      | Schedules at the lowest predicted carbon point without error variance.         |
| Carbon-Aware Baseline           | Standard heuristic baseline balancing average carbon and basic deadlines.      |
| Uncertainty-Aware (CarbonRoute) | Optimizes carbon while strictly bounding calibrated deadline violation risk.   |
| Oracle Reference Solver         | Theoretical benchmark using realized future info (not a production scheduler). |

## Uncertainty-Aware Decision Formulation

CarbonRoute does not simply choose the lowest predicted carbon value. It considers uncertainty and deadline risk when comparing feasible schedules.

### Scenario A: High Confidence Forecast

- > Forecast Confidence: High (Low Variance)
- > Action: Wait for deep carbon valley
- > Carbon Benefit: Significant reduction
- > **Deadline Violation Risk: Calibrated  $\leq 5\%$**
- > Recommendation: Approve delayed dispatch

### Scenario B: Low Confidence Forecast

- > Forecast Confidence: Low (High Uncertainty)
- > Action: Shift back to safer earlier window
- > Carbon Tradeoff: Higher carbon acceptable
- > **Deadline Violation Risk: Protected  $< 10\%$**
- > Recommendation: Prevent risky delay

## 7. 17-Week Roadmap

# 17-Week Software Engineering Roadmap (UCS503)

|             |                                                                           |
|-------------|---------------------------------------------------------------------------|
| WEEKS 1-2   | Requirements, research, initial simulator prototype, platform setup       |
| WEEKS 3-4   | Simulator models and workload generation (discrete-event trace engine)    |
| WEEKS 5-6   | Baseline scheduling policies (Immediate, EDF, Cost-Aware, Oracle)         |
| WEEKS 7-8   | Deterministic Carbon-Aware scheduler and trace integration                |
| WEEKS 9-10  | Forecast uncertainty and multi-horizon error modeling                     |
| WEEKS 11-12 | Uncertainty-aware scheduler and deadline-risk calibration                 |
| WEEK 13     | Stress testing and edge-case validation under volatility                  |
| WEEK 14     | API and interactive dashboard integration                                 |
| WEEK 15     | Large-scale repeated experiments and reproducible benchmark               |
| WEEK 16     | Containerized workload connector (Kubernetes Jobs), testing & deployment  |
| WEEK 17     | Final integration, evaluation, documentation, and university presentation |

## 8. Evaluation Strategy

# Rigorous Evaluation Metrics (Planned)

All metrics will be empirically evaluated during benchmarking phases (Zero fake results)

- > 1. Carbon Emissions: Cumulative gCO<sub>2</sub>eq consumed per batch execution cycle.
- > 2. Cloud Cost: Infrastructure and electricity cost incurred across regions.
- > 3. Job Delay / Makespan: Average and 95th-percentile execution wait time.
- > 4. Deadline Violation Rate: Percentage of batch jobs failing to complete before deadline.
- > 5. Risk Calibration: Brier score and Expected Calibration Error (ECE) for predicted vs realized risk.
- > 6. Oracle Regret: Distance from theoretical optimal schedule computed by Oracle.
- > 7. Runtime & Resource Utilization: Algorithmic solve overhead and compute efficiency.

## CarbonRoute Research Team & Deliverables

### Yuvika Nagpal

#### Core Responsibilities:

- > Simulation
- > Data
- > Workload Modelling

### Kumkum Gupta

#### Core Responsibilities:

- > Scheduling Algorithms
- > Uncertainty
- > Risk Calibration

### Aaneya Sabharwal

#### Core Responsibilities:

- > Backend
- > Dashboard
- > Deployment
- > Integration

Shared Responsibilities: Testing, documentation, evaluation, and presentation are shared team responsibilities.